

Praktikum 7: Main Memory

Informasi Praktikum

- **Mata Kuliah:** Sistem Operasi
 - **Sistem Operasi:** Linux Ubuntu
 - **Bahasa Pemrograman:** C
 - **Dosen Pengampu:** Triawan Adi Cahyanto, M.Kom
 - **Asisten Dosen:** Atidhira Habibillah dan Taqiyyuddin
-

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep dasar manajemen memori dalam sistem operasi
 2. Mengimplementasikan algoritma alokasi memori berurutan (contiguous)
 3. Memahami dan mensimulasikan mekanisme paging
 4. Mengimplementasikan struktur page table
 5. Memahami konsep swapping dan implementasinya
 6. Menganalisis arsitektur memori Intel dan ARM
-

Persiapan Praktikum

A. Instalasi Tools yang Diperlukan

Buka terminal dan jalankan perintah berikut:

```
# Update repository
sudo apt update

# Install GCC compiler
sudo apt install build-essential -y

# Install text editor (pilih salah satu)
sudo apt install vim nano gedit -y

# Install tools debugging (opsional)
sudo apt install gdb valgrind -y

# Verifikasi instalasi
gcc --version
```

B. Persiapan Direktori Kerja

```
# Buat direktori untuk praktikum
mkdir -p ~/praktikum-memory
cd ~/praktikum-memory

# Buat subdirektori untuk setiap bagian
mkdir background contiguous-allocation paging page-table swapping
architecture
```

Topik 1: Konsep Dasar Memori

1.1 Teori Singkat

Memori utama (main memory) adalah tempat penyimpanan yang dapat diakses langsung oleh CPU. Dalam sistem operasi, manajemen memori meliputi:

- **Address Binding:** Proses pemetaan alamat logis ke alamat fisik
- **Logical vs Physical Address:** Alamat yang digunakan program vs alamat sebenarnya di RAM
- **Memory Protection:** Melindungi area memori antar proses

1.2 Praktik 1: Memahami Address Space

File: **background/address_demo.c**

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;           // Data segment
int uninitialized_var;         // BSS segment

void print_addresses() {
    int local_var = 50;        // Stack
    int *heap_var = (int*)malloc(sizeof(int)); // Heap
    *heap_var = 75;

    printf("=== MEMORY ADDRESS LAYOUT ===\n\n");
    printf("1. Text Segment (Code):\n");
    printf("    Function address: %p\n\n", (void*)print_addresses);

    printf("2. Data Segment (Initialized Global):\n");
    printf("    global_var address: %p, value: %d\n\n",
        (void*)&global_var, global_var);

    printf("3. BSS Segment (Uninitialized Global):\n");
    printf("    uninitialized_var address: %p, value: %d\n\n",
        (void*)&uninitialized_var, uninitialized_var);

    printf("4. Heap Segment (Dynamic Allocation):\n");
    printf("    heap_var address: %p, value: %d\n\n",
        (void*)heap_var, *heap_var);
}
```

```
printf("5. Stack Segment (Local Variables):\n");
printf("    local_var address: %p, value: %d\n\n",
      (void*)&local_var, local_var);

free(heap_var);
}

int main() {
printf("Program dimulai...\n");
printf("PID: %d\n\n", getpid());

print_addresses();

printf("\nTekan Enter untuk melihat memory map...");
getchar();

// Perintah untuk melihat memory map
char cmd[100];
sprintf(cmd, "cat /proc/%d/maps", getpid());
system(cmd);

return 0;
}
```

Langkah Praktik:

```
# 1. Pindah ke direktori background
cd ~/praktikum-memory/background

# 2. Buat file dengan editor
nano address_demo.c
# Copy-paste kode di atas, simpan dengan Ctrl+O, Enter, Ctrl+X

# 3. Compile program
gcc -o address_demo address_demo.c

# 4. Jalankan program
./address_demo
```

Output yang Diharapkan:

- Alamat memori dari berbagai segment (text, data, bss, heap, stack)
- Memory map dari proses

Tulis jawaban pertanyaan berikut ini di laporan praktikum:

1. Bandingkan alamat memori dari setiap segment
2. Segment mana yang memiliki alamat tertinggi dan terendah?
3. Jalankan program beberapa kali, apakah alamat berubah? Mengapa?

Topik 2: Contiguous Memory Allocation

2.1 Teori Singkat

Alokasi memori berurutan membagi memori menjadi beberapa partisi. Ada tiga algoritma utama:

- **First-Fit:** Alokasi pada hole pertama yang cukup besar
- **Best-Fit:** Alokasi pada hole terkecil yang cukup besar
- **Worst-Fit:** Alokasi pada hole terbesar

2.2 Praktik 2: Simulasi Algoritma Alokasi Memori

File: `contiguous-allocation/memory_allocator.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_BLOCKS 10
#define MEMORY_SIZE 1000

typedef struct {
    int id;
    int size;
    int start_address;
    int is_allocated;
} MemoryBlock;

MemoryBlock memory[MAX_BLOCKS];
int block_count = 0;

void initialize_memory() {
    // Inisialisasi dengan satu block kosong besar
    memory[0].id = -1; // -1 untuk free block
    memory[0].size = MEMORY_SIZE;
    memory[0].start_address = 0;
    memory[0].is_allocated = 0;
    block_count = 1;
}

void display_memory() {
    printf("\n=== STATUS MEMORI ===\n");
    printf("%-10s %-10s %-15s %-10s\n",
           "Block", "Size", "Start Address", "Status");
    printf("-----\n");

    for (int i = 0; i < block_count; i++) {
        printf("%-10d %-10d %-15d %-10s\n",
               memory[i].id,
               memory[i].size,
               memory[i].start_address,
```

```

        memory[i].is_allocated ? "Allocated" : "Free");
    }
    printf("\n");
}

int first_fit(int process_id, int size) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            // Alokasi di block ini
            int old_size = memory[i].size;
            int old_start = memory[i].start_address;

            memory[i].id = process_id;
            memory[i].size = size;
            memory[i].is_allocated = 1;

            // Jika ada sisa, buat block baru
            if (old_size > size) {
                // Geser block-block setelahnya
                for (int j = block_count; j > i + 1; j--) {
                    memory[j] = memory[j-1];
                }

                memory[i+1].id = -1;
                memory[i+1].size = old_size - size;
                memory[i+1].start_address = old_start + size;
                memory[i+1].is_allocated = 0;
                block_count++;
            }

            printf("First-Fit: Process %d (%d bytes) dialokasikan di\n",
                address %d\n",
                    process_id, size, old_start);
            return 1;
        }
    }
    printf("First-Fit: Gagal mengalokasikan Process %d (%d bytes)\n",
        process_id, size);
    return 0;
}

int best_fit(int process_id, int size) {
    int best_idx = -1;
    int smallest_hole = MEMORY_SIZE + 1;

    // Cari hole terkecil yang cukup
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            if (memory[i].size < smallest_hole) {
                smallest_hole = memory[i].size;
                best_idx = i;
            }
        }
    }
}

```

```
if (best_idx == -1) {
    printf("Best-Fit: Gagal mengalokasikan Process %d (%d bytes)\n",
           process_id, size);
    return 0;
}

// Alokasi di best_idx
int old_size = memory[best_idx].size;
int old_start = memory[best_idx].start_address;

memory[best_idx].id = process_id;
memory[best_idx].size = size;
memory[best_idx].is_allocated = 1;

if (old_size > size) {
    for (int j = block_count; j > best_idx + 1; j--) {
        memory[j] = memory[j-1];
    }

    memory[best_idx+1].id = -1;
    memory[best_idx+1].size = old_size - size;
    memory[best_idx+1].start_address = old_start + size;
    memory[best_idx+1].is_allocated = 0;
    block_count++;
}

printf("Best-Fit: Process %d (%d bytes) dialokasikan di address %d\n",
       process_id, size, old_start);
return 1;
}

int worst_fit(int process_id, int size) {
    int worst_idx = -1;
    int largest_hole = -1;

    // Cari hole terbesar
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            if (memory[i].size > largest_hole) {
                largest_hole = memory[i].size;
                worst_idx = i;
            }
        }
    }

    if (worst_idx == -1) {
        printf("Worst-Fit: Gagal mengalokasikan Process %d (%d bytes)\n",
               process_id, size);
        return 0;
    }

    // Alokasi di worst_idx
    int old_size = memory[worst_idx].size;
```

```

    int old_start = memory[worst_idx].start_address;

    memory[worst_idx].id = process_id;
    memory[worst_idx].size = size;
    memory[worst_idx].is_allocated = 1;

    if (old_size > size) {
        for (int j = block_count; j > worst_idx + 1; j--) {
            memory[j] = memory[j-1];
        }

        memory[worst_idx+1].id = -1;
        memory[worst_idx+1].size = old_size - size;
        memory[worst_idx+1].start_address = old_start + size;
        memory[worst_idx+1].is_allocated = 0;
        block_count++;
    }

    printf("Worst-Fit: Process %d (%d bytes) dialokasikan di address\n",
        process_id, size, old_start);
    return 1;
}

void deallocate(int process_id) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].id == process_id && memory[i].is_allocated) {
            memory[i].id = -1;
            memory[i].is_allocated = 0;
            printf("Process %d berhasil di-dealokasi\n", process_id);

            // Merge dengan block sebelah jika free
            // Merge dengan block setelahnya
            if (i + 1 < block_count && !memory[i+1].is_allocated) {
                memory[i].size += memory[i+1].size;
                for (int j = i + 1; j < block_count - 1; j++) {
                    memory[j] = memory[j+1];
                }
                block_count--;
            }

            // Merge dengan block sebelumnya
            if (i > 0 && !memory[i-1].is_allocated) {
                memory[i-1].size += memory[i].size;
                for (int j = i; j < block_count - 1; j++) {
                    memory[j] = memory[j+1];
                }
                block_count--;
            }

            return;
        }
    }

    printf("Process %d tidak ditemukan\n", process_id);
}

```

```

}

void test_algorithm(int choice) {
    initialize_memory();

    printf("\n=====\\n");
    printf("Testing: ");
    switch(choice) {
        case 1: printf("FIRST-FIT ALGORITHM\\n"); break;
        case 2: printf("BEST-FIT ALGORITHM\\n"); break;
        case 3: printf("WORST-FIT ALGORITHM\\n"); break;
    }
    printf("=====\\n");

    // Test case: alokasi beberapa process
    int processes[][2] = {
        {1, 100}, // Process 1: 100 bytes
        {2, 200}, // Process 2: 200 bytes
        {3, 300}, // Process 3: 300 bytes
        {4, 150}, // Process 4: 150 bytes
    };

    for (int i = 0; i < 4; i++) {
        switch(choice) {
            case 1: first_fit(processes[i][0], processes[i][1]); break;
            case 2: best_fit(processes[i][0], processes[i][1]); break;
            case 3: worst_fit(processes[i][0], processes[i][1]); break;
        }
    }

    display_memory();

    // Dealokasi process 2
    printf("Dealokasi Process 2...\\n");
    deallocate(2);
    display_memory();

    // Alokasi process baru
    printf("Alokasi Process 5 (180 bytes)...\\n");
    switch(choice) {
        case 1: first_fit(5, 180); break;
        case 2: best_fit(5, 180); break;
        case 3: worst_fit(5, 180); break;
    }

    display_memory();
}

int main() {
    int choice;

    while(1) {
        printf("\\n=== SIMULASI CONTIGUOUS MEMORY ALLOCATION ===\\n");
        printf("1. First-Fit Algorithm\\n");
    }
}

```



```
printf("2. Best-Fit Algorithm\n");
printf("3. Worst-Fit Algorithm\n");
printf("4. Test Semua Algoritma\n");
printf("5. Exit\n");
printf("Pilihan: ");
scanf("%d", &choice);

if (choice == 5) break;

if (choice == 4) {
    for (int i = 1; i <= 3; i++) {
        test_algorithm(i);
        printf("\nTekan Enter untuk lanjut...");
        getchar(); getchar();
    }
} else if (choice >= 1 && choice <= 3) {
    test_algorithm(choice);
}

return 0;
}
```

Langkah Praktik:

```
# 1. Pindah ke direktori
cd ~/praktikum-memory/contiguous-allocation

# 2. Buat file
nano memory_allocator.c

# 3. Compile
gcc -o memory_allocator memory_allocator.c

# 4. Jalankan
./memory_allocator
```

Tulis jawaban pertanyaan berikut ini di laporan praktikum:

1. Jalankan semua algoritma (pilih menu 4)
 2. Bandingkan hasil dari ketiga algoritma
 3. Algoritma mana yang paling efisien? Jelaskan!
 4. Apa yang terjadi setelah dealokasi dan alokasi ulang?
-

Topik 3: Paging

3.1 Teori Singkat

Paging membagi memori fisik menjadi frame berukuran tetap dan memori logis menjadi page dengan ukuran sama. Keuntungan:

- Menghilangkan external fragmentation
- Tidak perlu alokasi berurutan

3.2 Praktik 3: Simulasi Paging System

File: **paging/paging_simulator.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

#define FRAME_SIZE 256          // Ukuran frame dalam bytes
#define NUM_FRAMES 16          // Jumlah frame di memori fisik
#define PAGE_SIZE FRAME_SIZE    // Page size = Frame size

typedef struct {
    int page_number;
    int frame_number;
    int valid;                // 1 jika page ada di memori
    int reference_bit;        // Untuk algoritma page replacement
    int modify_bit;          // Dirty bit
} PageTableEntry;

typedef struct {
    int process_id;
    int occupied;
    int page_number;
} Frame;

Frame physical_memory[NUM_FRAMES];
PageTableEntry page_table[100]; // Max 100 pages per process
int num_pages = 0;

void initialize_system() {
    // Inisialisasi physical memory
    for (int i = 0; i < NUM_FRAMES; i++) {
        physical_memory[i].occupied = 0;
        physical_memory[i].process_id = -1;
        physical_memory[i].page_number = -1;
    }

    // Inisialisasi page table
    for (int i = 0; i < 100; i++) {
        page_table[i].page_number = i;
    }
}
```

```

        page_table[i].frame_number = -1;
        page_table[i].valid = 0;
        page_table[i].reference_bit = 0;
        page_table[i].modify_bit = 0;
    }

    printf("  Sistem paging diinisialisasi\n");
    printf("  Frame size: %d bytes\n", FRAME_SIZE);
    printf("  Number of frames: %d\n", NUM_FRAMES);
    printf("  Total physical memory: %d bytes\n\n", FRAME_SIZE *
NUM_FRAMES);
}

void display_page_table() {
    printf("\n=== PAGE TABLE ===\n");
    printf("%-12s %-15s %-10s %-12s %-12s\n",
        "Page", "Frame", "Valid", "Referenced", "Modified");
    printf("-----\n");

    for (int i = 0; i < num_pages; i++) {
        printf("%-12d %-15d %-10s %-12s %-12s\n",
            page_table[i].page_number,
            page_table[i].frame_number,
            page_table[i].valid ? "Yes" : "No",
            page_table[i].reference_bit ? "Yes" : "No",
            page_table[i].modify_bit ? "Yes" : "No");
    }
    printf("\n");
}

void display_physical_memory() {
    printf("=== PHYSICAL MEMORY (FRAMES) ===\n");
    printf("%-10s %-10s %-15s\n", "Frame", "Status", "Page Number");
    printf("-----\n");

    for (int i = 0; i < NUM_FRAMES; i++) {
        printf("%-10d %-10s", i, physical_memory[i].occupied ? "Occupied"
: "Free");
        if (physical_memory[i].occupied) {
            printf("%-15d", physical_memory[i].page_number);
        }
        printf("\n");
    }
    printf("\n");
}

int find_free_frame() {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (!physical_memory[i].occupied) {
            return i;
        }
    }
    return -1; // Tidak ada frame kosong
}

```

```
}

int allocate_page(int page_num) {
    if (page_num >= 100) {
        printf("Error: Page number terlalu besar\n");
        return -1;
    }

    // Cek apakah page sudah dialokasi
    if (page_table[page_num].valid) {
        printf("Page %d sudah ada di frame %d\n",
            page_num, page_table[page_num].frame_number);
        page_table[page_num].reference_bit = 1;
        return page_table[page_num].frame_number;
    }

    // Cari frame kosong
    int frame = find_free_frame();
    if (frame == -1) {
        printf("Error: Tidak ada frame kosong (perlu page
replacement)\n");
        return -1;
    }

    // Alokasi page ke frame
    page_table[page_num].frame_number = frame;
    page_table[page_num].valid = 1;
    page_table[page_num].reference_bit = 1;

    physical_memory[frame].occupied = 1;
    physical_memory[frame].page_number = page_num;

    if (page_num >= num_pages) {
        num_pages = page_num + 1;
    }

    printf("Page %d dialokasikan ke frame %d\n", page_num, frame);
    return frame;
}

int translate_address(int logical_address) {
    int page_num = logical_address / PAGE_SIZE;
    int offset = logical_address % PAGE_SIZE;

    printf("\n--- Address Translation ---\n");
    printf("Logical address: %d\n", logical_address);
    printf("Page number: %d\n", page_num);
    printf("Offset: %d\n", offset);

    if (page_num >= num_pages || !page_table[page_num].valid) {
        printf("PAGE FAULT! Page %d tidak ada di memori\n", page_num);
        printf("Melakukan page allocation...\n");

        int frame = allocate_page(page_num);
```

```

        if (frame == -1) return -1;
    }

    int frame_num = page_table[page_num].frame_number;
    int physical_address = frame_num * FRAME_SIZE + offset;

    printf("Frame number: %d\n", frame_num);
    printf("Physical address: %d\n", physical_address);
    page_table[page_num].reference_bit = 1;

    return physical_address;
}

void access_memory(int logical_address, int write) {
    printf("\n=====\\n");
    printf("Memory Access: %s at logical address %d\\n",
        write ? "WRITE" : "READ", logical_address);
    printf("=====\\n");

    int physical_addr = translate_address(logical_address);

    if (physical_addr != -1) {
        printf("Akses berhasil di physical address: %d\\n", physical_addr);

        int page_num = logical_address / PAGE_SIZE;
        if (write) {
            page_table[page_num].modify_bit = 1;
            printf("  (Page %d ditandai sebagai modified)\\n", page_num);
        }
    }
}

void demo_scenario() {
    printf("\\n");
    printf("DEMO SCENARIO: PAGING SYSTEM\\n");
    printf("\\n\\n");

    // Scenario: Program dengan ukuran 2048 bytes (8 pages)
    printf("Skenario: Program berukuran 2048 bytes\\n");
    printf("Jumlah pages: %d pages\\n\\n", 2048 / PAGE_SIZE);

    // Akses beberapa alamat
    access_memory(0, 0);      // Read page 0
    access_memory(300, 1);    // Write page 1
    access_memory(512, 0);    // Read page 2
    access_memory(100, 0);    // Read page 0 lagi
    access_memory(1024, 1);   // Write page 4

    display_page_table();
    display_physical_memory();
}

int main() {

```

```
int choice, address, write;

initialize_system();

while(1) {
    printf("\n=== PAGING SIMULATOR ===\n");
    printf("1. Allocate Page\n");
    printf("2. Translate Logical Address\n");
    printf("3. Access Memory (Read)\n");
    printf("4. Access Memory (Write)\n");
    printf("5. Display Page Table\n");
    printf("6. Display Physical Memory\n");
    printf("7. Run Demo Scenario\n");
    printf("8. Reset System\n");
    printf("9. Exit\n");
    printf("Pilihan: ");
    scanf("%d", &choice);

    switch(choice) {
        case 1:
            printf("Masukkan page number: ");
            scanf("%d", &address);
            allocate_page(address);
            break;

        case 2:
            printf("Masukkan logical address: ");
            scanf("%d", &address);
            translate_address(address);
            break;

        case 3:
            printf("Masukkan logical address: ");
            scanf("%d", &address);
            access_memory(address, 0);
            break;

        case 4:
            printf("Masukkan logical address: ");
            scanf("%d", &address);
            access_memory(address, 1);
            break;

        case 5:
            display_page_table();
            break;

        case 6:
            display_physical_memory();
            break;

        case 7:
            demo_scenario();
            break;
```

```
        case 8:
            initialize_system();
            num_pages = 0;
            break;

        case 9:
            printf("Keluar dari program...\n");
            return 0;
    }

    return 0;
}
```

Langkah Praktik:

```
# 1. Pindah ke direktori
cd ~/praktikum-memory/paging

# 2. Buat file
nano paging_simulator.c

# 3. Compile
gcc -o paging_simulator paging_simulator.c -lm

# 4. Jalankan
./paging_simulator
```

Tulis jawaban pertanyaan berikut ini di laporan praktikum:

1. Jalankan demo scenario (pilih menu 7)
 2. Coba akses alamat logis: 0, 256, 512, 768
 3. Apa yang terjadi saat page fault?
 4. Hitung: Jika logical address = 1500, berapa page number dan offset-nya?
-

Topik 4: Structure of The Page Table

4.1 Teori Singkat

Struktur page table mempengaruhi efisiensi akses memori:

- **Hierarchical Paging:** Page table multi-level
- **Hashed Page Table:** Menggunakan hash function
- **Inverted Page Table:** Satu entry per frame (bukan per page)

4.2 Praktik 4: Two-Level Page Table

File: `page-table/two_level_paging.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Konfigurasi sistem
#define PAGE_SIZE 4096          // 4KB
#define VIRTUAL_ADDR_BITS 32   // 32-bit address
#define OFFSET_BITS 12         // log2(4096) = 12 bits
#define PAGE_NUMBER_BITS (VIRTUAL_ADDR_BITS - OFFSET_BITS) // 20 bits

// Two-level configuration
#define OUTER_PAGE_BITS 10      // 10 bits untuk outer page
#define INNER_PAGE_BITS 10      // 10 bits untuk inner page
// Total: 10 + 10 + 12 = 32 bits

typedef struct {
    int frame_number;
    int valid;
} InnerPageTableEntry;

typedef struct {
    InnerPageTableEntry *inner_table;
    int valid;
} OuterPageTableEntry;

OuterPageTableEntry *outer_page_table;
int num_outer_entries;
int num_inner_entries;

void initialize_two_level_paging() {
    num_outer_entries = (int)pow(2, OUTER_PAGE_BITS);
    num_inner_entries = (int)pow(2, INNER_PAGE_BITS);

    // Alokasi outer page table
    outer_page_table = (OuterPageTableEntry*)calloc(num_outer_entries,
    sizeof(OuterPageTableEntry));
```



```

printf("=== TWO-LEVEL PAGE TABLE INITIALIZED ===\n");
printf("Virtual address: %d bits\n", VIRTUAL_ADDR_BITS);
printf("Page size: %d bytes\n", PAGE_SIZE);
printf("Outer page bits: %d (entries: %d)\n",
        OUTER_PAGE_BITS, num_outer_entries);
printf("Inner page bits: %d (entries: %d)\n",
        INNER_PAGE_BITS, num_inner_entries);
printf("Offset bits: %d\n", OFFSET_BITS);
printf("\nMemory untuk outer table: %lu bytes\n",
        num_outer_entries * sizeof(OuterPageTableEntry));
printf("Memory untuk satu inner table: %lu bytes\n",
        num_inner_entries * sizeof(InnerPageTableEntry));
printf("=====\n\n");
}

void parse_virtual_address(unsigned int virtual_addr, int *outer, int
*inner, int *offset) {
    // Extract offset (12 bits terakhir)
    *offset = virtual_addr & ((1 << OFFSET_BITS) - 1);

    // Extract inner page number (10 bits berikutnya)
    *inner = (virtual_addr >> OFFSET_BITS) & ((1 << INNER_PAGE_BITS) - 1);

    // Extract outer page number (10 bits pertama)
    *outer = (virtual_addr >> (OFFSET_BITS + INNER_PAGE_BITS)) &
        ((1 << OUTER_PAGE_BITS) - 1);
}

int translate_address_two_level(unsigned int virtual_addr) {
    int outer_idx, inner_idx, offset;

    parse_virtual_address(virtual_addr, &outer_idx, &inner_idx, &offset);

    printf("\n--- Two-Level Address Translation ---\n");
    printf("Virtual Address: 0x%08X (%u)\n", virtual_addr, virtual_addr);
    printf("Outer Page Index: %d\n", outer_idx);
    printf("Inner Page Index: %d\n", inner_idx);
    printf("Offset: %d\n", offset);

    // Check outer page table
    if (!outer_page_table[outer_idx].valid) {
        printf("x Page Fault: Outer page table entry %d tidak valid\n",
outer_idx);
        return -1;
    }

    // Check inner page table
    InnerPageTableEntry *inner_table =
outer_page_table[outer_idx].inner_table;
    if (!inner_table[inner_idx].valid) {
        printf("x Page Fault: Inner page table entry %d tidak valid\n",
inner_idx);
        return -1;
    }
}

```

```

    // Calculate physical address
    int frame_number = inner_table[inner_idx].frame_number;
    int physical_addr = frame_number * PAGE_SIZE + offset;

    printf("Frame Number: %d\n", frame_number);
    printf("Physical Address: 0x%08X (%d)\n", physical_addr,
physical_addr);
    printf("✓ Translation successful!\n");

    return physical_addr;
}

void allocate_page_two_level(unsigned int virtual_addr, int frame_number)
{
    int outer_idx, inner_idx, offset;

    parse_virtual_address(virtual_addr, &outer_idx, &inner_idx, &offset);

    printf("\n--- Allocating Page ---\n");
    printf("Virtual Address: 0x%08X\n", virtual_addr);
    printf("Outer Index: %d, Inner Index: %d\n", outer_idx, inner_idx);
    printf("Assigning to Frame: %d\n", frame_number);

    // Alokasi inner table jika belum ada
    if (!outer_page_table[outer_idx].valid) {
        outer_page_table[outer_idx].inner_table =
            (InnerPageTableEntry*)calloc(num_inner_entries,
                                        sizeof(InnerPageTableEntry));
        outer_page_table[outer_idx].valid = 1;
        printf("✓ Inner page table created for outer index %d\n",
outer_idx);
    }

    // Set inner page table entry
    InnerPageTableEntry *inner_table =
outer_page_table[outer_idx].inner_table;
    inner_table[inner_idx].frame_number = frame_number;
    inner_table[inner_idx].valid = 1;

    printf("✓ Page allocated successfully!\n");
}

void display_page_table_two_level() {
    int total_valid = 0;
    int total_inner_tables = 0;

    printf("\n=== TWO-LEVEL PAGE TABLE STATUS ===\n");

    for (int i = 0; i < num_outer_entries; i++) {
        if (outer_page_table[i].valid) {
            total_inner_tables++;
            printf("\nOuter Entry %d: VALID\n", i);
            printf("  Inner Table Entries:\n");

```

```

        InnerPageTableEntry *inner = outer_page_table[i].inner_table;
        for (int j = 0; j < num_inner_entries; j++) {
            if (inner[j].valid) {
                total_valid++;
                printf("    [%d][%d] -> Frame %d\n", i, j,
inner[j].frame_number);
            }
        }
    }
}

printf("\n--- Summary ---\n");
printf("Active Outer Entries: %d / %d\n", total_inner_tables,
num_outer_entries);
printf("Total Valid Pages: %d\n", total_valid);
printf("Memory Overhead:\n");
printf("  Outer Table: %lu bytes\n",
        num_outer_entries * sizeof(OuterPageTableEntry));
printf("  Inner Tables: %lu bytes (x%d)\n",
        num_inner_entries * sizeof(InnerPageTableEntry),
total_inner_tables);
printf("  Total: %lu bytes\n",
        num_outer_entries * sizeof(OuterPageTableEntry) +
        total_inner_tables * num_inner_entries *
sizeof(InnerPageTableEntry));
printf("=====\n\n");
}

void calculate_overhead() {
    // Single-level page table
    int single_level_entries = (int)pow(2, PAGE_NUMBER_BITS);
    long single_level_size = single_level_entries *
sizeof(InnerPageTableEntry);

    // Two-level (best case: hanya 1 inner table)
    long two_level_min = num_outer_entries * sizeof(OuterPageTableEntry) +
        num_inner_entries * sizeof(InnerPageTableEntry);

    // Two-level (worst case: semua inner table dialokasi)
    long two_level_max = num_outer_entries * sizeof(OuterPageTableEntry) +
        num_outer_entries * num_inner_entries *
sizeof(InnerPageTableEntry);

    printf("\n=== MEMORY OVERHEAD COMPARISON ===\n");
    printf("Single-Level Page Table:\n");
    printf("  Entries: %d\n", single_level_entries);
    printf("  Size: %.2f MB\n", single_level_size / (1024.0 * 1024.0));

    printf("\nTwo-Level Page Table:\n");
    printf("  Best Case (1 inner table): %.2f KB\n", two_level_min /
1024.0);
    printf("  Worst Case (all inner tables): %.2f MB\n", two_level_max /
(1024.0 * 1024.0));
}

```

```

    printf("  Space Savings (best case): %.2f%%\n",
           (1 - (float)two_level_min / single_level_size) * 100);
    printf("=====\n\n");
}

void demo_two_level() {
    printf("\n");
    printf("=====\n");
    printf("  DEMO: TWO-LEVEL PAGING SYSTEM\n");
    printf("=====\n\n");

    // Alokasi beberapa pages
    printf("Scenario: Allocating pages for a program\n\n");

    allocate_page_two_level(0x00000000, 10); // Page 0 -> Frame 10
    allocate_page_two_level(0x00001000, 11); // Page 1 -> Frame 11
    allocate_page_two_level(0x00400000, 20); // Page di outer entry
berbeda
    allocate_page_two_level(0x00401000, 21);

    display_page_table_two_level();

    // Test translation
    printf("Testing Address Translation:\n");
    translate_address_two_level(0x00000500); // Should hit
    translate_address_two_level(0x00002000); // Should miss (page fault)
    translate_address_two_level(0x00400100); // Should hit

    calculate_overhead();
}

void cleanup_two_level() {
    for (int i = 0; i < num_outer_entries; i++) {
        if (outer_page_table[i].valid) {
            free(outer_page_table[i].inner_table);
        }
    }
    free(outer_page_table);
    printf("✓ Memory cleaned up\n");
}

int main() {
    int choice;
    unsigned int vaddr;
    int frame;

    initialize_two_level_paging();

    while(1) {
        printf("\n=== TWO-LEVEL PAGE TABLE SIMULATOR ===\n");
        printf("1. Allocate Page\n");
        printf("2. Translate Virtual Address\n");
        printf("3. Display Page Table\n");
        printf("4. Calculate Memory Overhead\n");
    }
}

```

```
printf("5. Run Demo Scenario\n");
printf("6. Exit\n");
printf("Pilihan: ");
scanf("%d", &choice);

switch(choice) {
    case 1:
        printf("Masukkan virtual address (hex, e.g., 0x1000): ");
        scanf("%x", &vaddr);
        printf("Masukkan frame number: ");
        scanf("%d", &frame);
        allocate_page_two_level(vaddr, frame);
        break;

    case 2:
        printf("Masukkan virtual address (hex, e.g., 0x1000): ");
        scanf("%x", &vaddr);
        translate_address_two_level(vaddr);
        break;

    case 3:
        display_page_table_two_level();
        break;

    case 4:
        calculate_overhead();
        break;

    case 5:
        demo_two_level();
        break;

    case 6:
        cleanup_two_level();
        printf("Keluar dari program...\n");
        return 0;

    default:
        printf("Pilihan tidak valid!\n");
}
}

return 0;
}
```

Topik 5: Swapping

5.1 Teori Singkat

Swapping adalah mekanisme memindahkan proses dari/ke disk storage ketika memori utama penuh.

Komponen:

- **Swap Out:** Memindahkan proses dari memory ke disk
- **Swap In:** Memindahkan proses dari disk ke memory
- **Backing Store:** Area di disk untuk menyimpan proses yang di-swap

5.2 Praktik 5: Simulasi Swapping Mechanism

File: `swapping/swap_simulator.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

#define MAX_PROCESSES 20
#define PHYSICAL_MEMORY 1024 // 1024 MB
#define SWAP_SPACE 2048 // 2048 MB

typedef enum {
    NEW,
    READY,
    RUNNING,
    SWAPPED,
    TERMINATED
} ProcessState;

typedef struct {
    int pid;
    char name[20];
    int size; // Ukuran proses dalam MB
    int priority; // 1-10 (10 = highest)
    ProcessState state;
    int in_memory;
    int swap_location; // Lokasi di swap space
    time_t load_time; // Waktu load ke memory
    time_t last_access; // Waktu akses terakhir
    int swap_count; // Jumlah di-swap
    int access_count; // Jumlah akses
} Process;

Process processes[MAX_PROCESSES];
int process_count = 0;
int used_memory = 0;
int used_swap = 0;
int total_swap_operations = 0;
```

```

void initialize_system() {
    process_count = 0;
    used_memory = 0;
    used_swap = 0;
    total_swap_operations = 0;

    for (int i = 0; i < MAX_PROCESSES; i++) {
        processes[i].pid = -1;
    }

    printf("\n");
    printf("    SWAPPING SYSTEM INITIALIZED    \n");
    printf("\n");
    printf("Physical Memory: %d MB\n", PHYSICAL_MEMORY);
    printf("Swap Space: %d MB\n\n", SWAP_SPACE);
}

void display_memory_status() {
    int in_memory_count = 0;
    int swapped_count = 0;

    printf("\n");
    printf("    MEMORY STATUS    \n");
    printf("\n");
    printf("Physical Memory: %d / %d MB (%.1f%% used)\n",
        used_memory, PHYSICAL_MEMORY,
        (float)used_memory/PHYSICAL_MEMORY * 100);
    printf("Swap Space: %d / %d MB (%.1f%% used)\n",
        used_swap, SWAP_SPACE,
        (float)used_swap/SWAP_SPACE * 100);
    printf("Total Swap Operations: %d\n\n", total_swap_operations);

    printf("%-5s %-15s %-8s %-10s %-12s %-8s %-10s\n",
        "PID", "Name", "Size", "Priority", "State", "Swaps",
        "Accesses");
    printf("-----\n");

    for (int i = 0; i < process_count; i++) {
        if (processes[i].pid != -1) {
            printf("%-5d %-15s %-8d %-10d %-12s %-8d %-10d\n",
                processes[i].pid,
                processes[i].name,
                processes[i].size,
                processes[i].priority,
                processes[i].state == READY ? "IN-MEMORY" :
                processes[i].state == SWAPPED ? "SWAPPED" : "OTHER",
                processes[i].swap_count,
                processes[i].access_count);

            if (processes[i].in_memory) in_memory_count++;
            else if (processes[i].state == SWAPPED) swapped_count++;
        }
    }
}

```

```

    }

    printf("\nProcesses in Memory: %d\n", in_memory_count);
    printf("Processes Swapped: %d\n", swapped_count);
    printf("Total Processes: %d\n\n", process_count);
}

int create_process(char *name, int size, int priority) {
    if (process_count >= MAX_PROCESSES) {
        printf("x Error: Maximum process limit reached\n");
        return -1;
    }

    if (size > PHYSICAL_MEMORY) {
        printf("x Error: Process too large for physical memory\n");
        return -1;
    }

    int idx = process_count;
    processes[idx].pid = process_count + 1;
    strncpy(processes[idx].name, name, 19);
    processes[idx].size = size;
    processes[idx].priority = priority;
    processes[idx].state = NEW;
    processes[idx].in_memory = 0;
    processes[idx].swap_location = -1;
    processes[idx].load_time = time(NULL);
    processes[idx].last_access = time(NULL);
    processes[idx].swap_count = 0;
    processes[idx].access_count = 0;

    process_count++;

    printf("✓ Process created: PID=%d, Name=%s, Size=%d MB,
Priority=%d\n",
        processes[idx].pid, name, size, priority);

    return processes[idx].pid;
}

int find_victim_fifo() {
    time_t oldest_time = time(NULL);
    int victim_idx = -1;

    for (int i = 0; i < process_count; i++) {
        if (processes[i].in_memory && processes[i].state == READY) {
            if (processes[i].load_time < oldest_time) {
                oldest_time = processes[i].load_time;
                victim_idx = i;
            }
        }
    }

    return victim_idx;
}

```



```

}

int find_victim_lru() {
    time_t oldest_access = time(NULL);
    int victim_idx = -1;

    for (int i = 0; i < process_count; i++) {
        if (processes[i].in_memory && processes[i].state == READY) {
            if (processes[i].last_access < oldest_access) {
                oldest_access = processes[i].last_access;
                victim_idx = i;
            }
        }
    }

    return victim_idx;
}

int find_victim_priority() {
    int lowest_priority = 11;
    int victim_idx = -1;

    for (int i = 0; i < process_count; i++) {
        if (processes[i].in_memory && processes[i].state == READY) {
            if (processes[i].priority < lowest_priority) {
                lowest_priority = processes[i].priority;
                victim_idx = i;
            }
        }
    }

    return victim_idx;
}

void swap_out(int idx, char *reason) {
    if (idx < 0 || idx >= process_count || !processes[idx].in_memory) {
        printf("x Invalid swap out operation\n");
        return;
    }

    printf("\n→ Swapping OUT Process %d (%s)\n",
           processes[idx].pid, processes[idx].name);
    printf("  Reason: %s\n", reason);
    printf("  Size: %d MB\n", processes[idx].size);

    // Simulasi waktu swap (proportional to size)
    usleep(processes[idx].size * 1000); // microseconds

    used_memory -= processes[idx].size;
    used_swap += processes[idx].size;
    processes[idx].in_memory = 0;
    processes[idx].state = SWAPPED;
    processes[idx].swap_location = used_swap;
    processes[idx].swap_count++;
}

```

```

        total_swap_operations++;

        printf("  ✓ Swapped to disk location: %d\n",
processes[idx].swap_location);
        printf("  Memory freed: %d MB (Available: %d MB)\n",
            processes[idx].size, PHYSICAL_MEMORY - used_memory);
    }

int swap_in(int idx) {
    if (idx < 0 || idx >= process_count || processes[idx].in_memory) {
        printf("x Invalid swap in operation\n");
        return -1;
    }

    printf("\n← Swapping IN Process %d (%s)\n",
        processes[idx].pid, processes[idx].name);
    printf("  Size: %d MB\n", processes[idx].size);

    // Cek apakah ada cukup memori
    if (used_memory + processes[idx].size > PHYSICAL_MEMORY) {
        printf("  x Not enough memory (need %d MB, available %d MB)\n",
            processes[idx].size, PHYSICAL_MEMORY - used_memory);
        return -1;
    }

    // Simulasi waktu swap (proportional to size)
    usleep(processes[idx].size * 1000);

    used_memory += processes[idx].size;
    used_swap -= processes[idx].size;
    processes[idx].in_memory = 1;
    processes[idx].state = READY;
    processes[idx].load_time = time(NULL);
    processes[idx].last_access = time(NULL);
    total_swap_operations++;

    printf("  ✓ Loaded to memory from location: %d\n",
        processes[idx].swap_location);
    printf("  Memory used: %d / %d MB\n", used_memory, PHYSICAL_MEMORY);

    return 0;
}

int load_process(int pid, int strategy) {
    int idx = -1;
    for (int i = 0; i < process_count; i++) {
        if (processes[i].pid == pid) {
            idx = i;
            break;
        }
    }

    if (idx == -1) {
        printf("x Process %d not found\n", pid);
    }
}

```

```

        return -1;
    }

    if (processes[idx].in_memory) {
        printf("✓ Process %d already in memory\n", pid);
        return 0;
    }

    // Jika tidak cukup memory, swap out victim
    while (used_memory + processes[idx].size > PHYSICAL_MEMORY) {
        int victim;
        char strategy_name[20];

        switch(strategy) {
            case 1: // FIFO
                victim = find_victim_fifo();
                strcpy(strategy_name, "FIFO");
                break;
            case 2: // LRU
                victim = find_victim_lru();
                strcpy(strategy_name, "LRU");
                break;
            case 3: // Priority
                victim = find_victim_priority();
                strcpy(strategy_name, "Priority-based");
                break;
            default:
                victim = find_victim_fifo();
                strcpy(strategy_name, "FIFO");
        }

        if (victim == -1) {
            printf("x No victim found to swap out\n");
            return -1;
        }

        char reason[100];
        sprintf(reason, "Make room using %s strategy", strategy_name);
        swap_out(victim, reason);
    }

    // Swap in process
    return swap_in(idx);
}

void access_process(int pid) {
    for (int i = 0; i < process_count; i++) {
        if (processes[i].pid == pid) {
            if (!processes[i].in_memory) {
                printf("△ Process %d not in memory, loading...\n", pid);
                load_process(pid, 2); // Use LRU
            } else {
                printf("✓ Accessing Process %d (%s)\n",
                    pid, processes[i].name);
            }
        }
    }
}

```

```

        processes[i].last_access = time(NULL);
        processes[i].access_count++;
    }
    return;
}
}
printf("x Process %d not found\n", pid);
}

void demo_scenario() {
    printf("\n");
    printf("DEMO SCENARIO: SWAPPING SYSTEM\n");
    printf("\n\n");

    // Create processes
    create_process("Chrome", 300, 8);
    create_process("VSCode", 250, 7);
    create_process("Firefox", 280, 6);
    create_process("Spotify", 150, 5);
    create_process("Terminal", 50, 9);
    create_process("Docker", 400, 7);

    printf("\n--- Loading processes using LRU strategy ---\n");

    // Load beberapa process
    load_process(1, 2); // Chrome
    load_process(2, 2); // VSCode
    load_process(3, 2); // Firefox
    load_process(5, 2); // Terminal

    display_memory_status();

    printf("\n--- Accessing processes ---\n");
    sleep(1);
    access_process(1); // Chrome
    sleep(1);
    access_process(5); // Terminal

    printf("\n--- Loading large process (Docker) ---\n");
    load_process(6, 2); // Docker - will trigger swapping

    display_memory_status();

    printf("\n--- Accessing swapped process ---\n");
    access_process(3); // Firefox (might be swapped)

    display_memory_status();
}

int main() {
    int choice, pid, size, priority, strategy;
    char name[20];

```

```
initialize_system();

while(1) {
    printf("\n===== \n");
    printf("      SWAPPING SIMULATOR MENU\n");
    printf("===== \n");
    printf("1.  Create Process\n");
    printf("2.  Load Process (FIFO)\n");
    printf("3.  Load Process (LRU)\n");
    printf("4.  Load Process (Priority-based)\n");
    printf("5.  Access Process\n");
    printf("6.  Swap Out Process\n");
    printf("7.  Display Memory Status\n");
    printf("8.  Run Demo Scenario\n");
    printf("9.  Reset System\n");
    printf("10. Exit\n");
    printf("===== \n");
    printf("Pilihan: ");
    scanf("%d", &choice);

    switch(choice) {
        case 1:
            printf("Process name: ");
            scanf("%s", name);
            printf("Size (MB): ");
            scanf("%d", &size);
            printf("Priority (1-10): ");
            scanf("%d", &priority);
            create_process(name, size, priority);
            break;

        case 2:
            printf("Process PID: ");
            scanf("%d", &pid);
            load_process(pid, 1); // FIFO
            break;

        case 3:
            printf("Process PID: ");
            scanf("%d", &pid);
            load_process(pid, 2); // LRU
            break;

        case 4:
            printf("Process PID: ");
            scanf("%d", &pid);
            load_process(pid, 3); // Priority
            break;

        case 5:
            printf("Process PID: ");
            scanf("%d", &pid);
            access_process(pid);
            break;
    }
}
```

```
        case 6:
            printf("Process PID: ");
            scanf("%d", &pid);
            for (int i = 0; i < process_count; i++) {
                if (processes[i].pid == pid) {
                    swap_out(i, "Manual swap out");
                    break;
                }
            }
            break;

        case 7:
            display_memory_status();
            break;

        case 8:
            demo_scenario();
            break;

        case 9:
            initialize_system();
            break;

        case 10:
            printf("Exiting...\n");
            return 0;

        default:
            printf("Invalid choice!\n");
    }
}

return 0;
}
```

Langkah Praktik:

```
# 1. Pindah ke direktori
cd ~/praktikum-memory/swapping

# 2. Buat file
nano swap_simulator.c

# 3. Compile
gcc -o swap_simulator swap_simulator.c

# 4. Jalankan
./swap_simulator
```

Tulis jawaban pertanyaan berikut ini di laporan praktikum:

1. Jalankan demo scenario (menu 8)
2. Bandingkan hasil FIFO, LRU, dan Priority-based strategy
3. Strategi mana yang menghasilkan swap operation paling sedikit?
4. Apa yang terjadi ketika loading process yang lebih besar dari memory yang tersedia?

Topik 6: Example Intel 32 & 64-Bit Architectures

6.1 Teori Singkat

Intel IA-32 (32-bit):

- Address space: 4 GB (2^{32} bytes)
- Page size: 4 KB atau 4 MB
- Two-level paging (4KB pages) atau single-level (4MB pages)

Intel x86-64 (64-bit):

- Address space: 256 TB (2^{48} bytes praktis)
- Page size: 4 KB, 2 MB, atau 1 GB
- Four-level paging

6.2 Praktik 6: Analisis Arsitektur Intel

File: **architecture/intel_analysis.sh**

```
#!/bin/bash

echo ""
echo " INTEL ARCHITECTURE ANALYSIS "
echo ""
echo ""

# 1. CPU Information
echo "=== CPU INFORMATION ==="
lscpu | grep -E "Architecture|CPU op-mode|Byte Order|Address sizes|CPU\
(s\)|Thread|Core|Model name"
echo ""

# 2. Page Size
echo "=== PAGE SIZE ==="
PAGE_SIZE=$(getconf PAGE_SIZE)
echo "Page Size: $PAGE_SIZE bytes (($((PAGE_SIZE/1024)) KB)"
echo ""

# 3. Virtual Memory Info
echo "=== VIRTUAL MEMORY ==="
cat /proc/meminfo | grep -E
"MemTotal|MemFree|MemAvailable|SwapTotal|SwapFree|Dirty|Writeback"
echo ""
```

```

# 4. Address Space Limits
echo "=== ADDRESS SPACE LIMITS ==="
ADDR_BITS=$(lscpu | grep "Address sizes" | awk '{print $3}' | cut -d',' -f1)
echo "Virtual Address Bits: $ADDR_BITS"
PHYS_BITS=$(lscpu | grep "Address sizes" | awk '{print $5}')
echo "Physical Address Bits: $PHYS_BITS"
echo ""

# 5. TLB Information (jika cpuid tersedia)
echo "=== TLB INFORMATION ==="
if command -v cpuid &> /dev/null; then
    cpuid | grep -i tlb | head -10
else
    echo "cpuid not installed. Install with: sudo apt install cpuid"
fi
echo ""

# 6. Cache Hierarchy
echo "=== CACHE HIERARCHY ==="
lscpu | grep -i cache
echo ""

# 7. Huge Pages
echo "=== HUGE PAGES SUPPORT ==="
cat /proc/meminfo | grep -i huge
echo ""

# 8. Memory Map Example
echo "=== MEMORY MAP OF THIS SCRIPT ==="
cat /proc/$$/maps | head -20
echo "... (truncated)"
echo ""

# 9. Calculate Virtual Address Space
echo "=== CALCULATED VALUES ==="
if [ ! -z "$ADDR_BITS" ]; then
    VIRT_SPACE=$((2**ADDR_BITS))
    echo "Maximum Virtual Address Space: 2^$ADDR_BITS bytes"

    if [ $ADDR_BITS -eq 32 ]; then
        echo "    = 4 GB"
    elif [ $ADDR_BITS -eq 48 ]; then
        echo "    = 256 TB"
    else
        TB=$((VIRT_SPACE / 1024 / 1024 / 1024 / 1024))
        echo "    ≈ $TB TB"
    fi
fi
echo ""

# 10. Paging Structure
echo "=== PAGING STRUCTURE ==="

```



```
ARCH=$(lscpu | grep "Architecture" | awk '{print $2}')  
if [[ $ARCH == *"64"* ]]; then  
    echo "Architecture: 64-bit (x86-64)"  
    echo "Paging Levels: 4-level paging"  
    echo " - PML4 (Page Map Level 4)"  
    echo " - PDPT (Page Directory Pointer Table)"  
    echo " - PD (Page Directory)"  
    echo " - PT (Page Table)"  
    echo ""  
    echo "Address Breakdown (for 4KB pages):"  
    echo " Bits 47-39: PML4 offset (9 bits)"  
    echo " Bits 38-30: PDPT offset (9 bits)"  
    echo " Bits 29-21: PD offset (9 bits)"  
    echo " Bits 20-12: PT offset (9 bits)"  
    echo " Bits 11-0: Page offset (12 bits)"  
else  
    echo "Architecture: 32-bit (IA-32)"  
    echo "Paging Levels: 2-level paging"  
    echo " - Page Directory"  
    echo " - Page Table"  
    echo ""  
    echo "Address Breakdown (for 4KB pages):"  
    echo " Bits 31-22: Page Directory offset (10 bits)"  
    echo " Bits 21-12: Page Table offset (10 bits)"  
    echo " Bits 11-0: Page offset (12 bits)"  
fi  
echo ""  
  
echo "
```

Langkah Praktik:

```
# 1. Pindah ke direktori
cd ~/praktikum-memory/architecture

# 2. Buat file
nano intel_analysis.sh

# 3. Berikan permission execute
chmod +x intel_analysis.sh

# 4. Jalankan script
./intel_analysis.sh

# 5. Simpan output
./intel_analysis.sh > intel_analysis_output.txt
```

Tulis jawaban pertanyaan berikut ini di laporan praktikum:

1. Dokumentasikan semua output dari script
 2. Hitung berapa jumlah entries di setiap level page table
 3. Berapa total memory overhead untuk page table jika semua virtual address space digunakan?
 4. Bandingkan dengan hasil perhitungan teoritis
-

Topik 7: Example ARMv8 Architecture

7.1 Teori Singkat

ARMv8 Architecture:

- Mendukung 32-bit (AArch32) dan 64-bit (AArch64)
- Virtual address: 48-bit atau 52-bit
- Page size: 4KB, 16KB, atau 64KB
- Multi-level translation table (hingga 4 level)

7.2 Praktik 7: Perbandingan Arsitektur

File: `architecture/architecture_comparison.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

// Struktur untuk menyimpan spesifikasi arsitektur memori
typedef struct {
    char name[30];
    int address_bits;
    int page_size;
    int levels;
    int bits_per_level[4]; // Menyimpan pembagian bit untuk setiap level
    page table
} ArchitectureSpec;

// Fungsi untuk mencetak detail arsitektur
void print_architecture_details(ArchitectureSpec *arch) {
    printf("\n");
    printf("    %s\n", arch->name);
    printf("    \n\n");

    printf("Address Space: %d bits\n", arch->address_bits);

    // Hitung Max Virtual Memory
    printf("Maximum Virtual Memory: ");
    if (arch->address_bits == 32) {
        printf("4 GB\n");
    } else if (arch->address_bits == 48) {
        printf("256 TB\n");
    } else {
        printf("2^%d bytes\n", arch->address_bits);
    }
}
```

```

    printf("\nPage Size: %d bytes (%d KB)\n", arch->page_size, arch->page_size / 1024);

    int offset_bits = (int)log2(arch->page_size);
    printf("Offset Bits: %d bits\n", offset_bits);

    printf("\nPaging Levels: %d\n", arch->levels);

    int total_table_bits = 0;
    for (int i = 0; i < arch->levels; i++) {
        if (arch->bits_per_level[i] > 0) {
            printf("  Level %d: %d bits (%d entries)\n",
                i + 1,
                arch->bits_per_level[i],
                (int)pow(2, arch->bits_per_level[i]));
            total_table_bits += arch->bits_per_level[i];
        }
    }

    printf("\nAddress Breakdown:\n");
    int current_bit = arch->address_bits - 1;
    for (int i = 0; i < arch->levels; i++) {
        if (arch->bits_per_level[i] > 0) {
            printf("  Bits %d-%d: Level %d index (%d bits)\n",
                current_bit,
                current_bit - arch->bits_per_level[i] + 1,
                i + 1,
                arch->bits_per_level[i]);
            current_bit -= arch->bits_per_level[i];
        }
    }
    printf("  Bits %d-0: Page offset (%d bits)\n", offset_bits - 1, offset_bits);

    // Simulasi Memory Overhead (Worst Case)
    // Catatan: Ini menghitung ukuran jika tabel dialokasikan PENUH
    (teoritis)
    printf("\nMemory Overhead (worst case - theoretical max entries):\n");
    double total_entries = 1.0;

    for (int i = 0; i < arch->levels; i++) {
        if (arch->bits_per_level[i] > 0) {
            long long entries_this_level = (long long)pow(2, arch->bits_per_level[i]);
            total_entries *= entries_this_level;

            // Asumsi 8 bytes per entry (64-bit architecture standard)
            // Untuk IA-32 biasanya 4 bytes, tapi kita generalisir untuk
            perbandingan
            double size_bytes = total_entries * 8;

            printf("  Level %d Cumulative Size: ", i + 1);

```

```

        if (size_bytes < 1024) {
            printf("%.0f bytes\n", size_bytes);
        } else if (size_bytes < 1024 * 1024) {
            printf("%.2f KB\n", size_bytes / 1024.0);
        } else if (size_bytes < 1024.0 * 1024 * 1024) {
            printf("%.2f MB\n", size_bytes / (1024.0 * 1024));
        } else if (size_bytes < 1024.0 * 1024 * 1024 * 1024) {
            printf("%.2f GB\n", size_bytes / (1024.0 * 1024 * 1024));
        } else {
            printf("%.2f TB\n", size_bytes / (1024.0 * 1024 * 1024 *
1024));
        }
    }
}
printf("\n");
}

void compare_architectures() {
    ArchitectureSpec ia32 = {
        "Intel IA-32 (32-bit)",
        32,
        4096, // 4KB
        2,
        {10, 10, 0, 0}
    };

    ArchitectureSpec x86_64 = {
        "Intel x86-64 (64-bit)",
        48,
        4096, // 4KB
        4,
        {9, 9, 9, 9}
    };

    ArchitectureSpec armv8_4k = {
        "ARMv8 with 4KB pages",
        48,
        4096,
        4,
        {9, 9, 9, 9}
    };

    ArchitectureSpec armv8_16k = {
        "ARMv8 with 16KB pages",
        48,
        16384, // 16KB
        4,
        {1, 11, 11, 11} // Distribusi bit berbeda karena offset lebih
besar
    };

    ArchitectureSpec armv8_64k = {
        "ARMv8 with 64KB pages",
        48,

```

```

        65536, // 64KB
        3,
        {5, 13, 13, 0} // Hanya butuh 3 level karena offset mencakup 16
bit
};

printf("\n");
printf("    ARCHITECTURE COMPARISON\n");
printf("\n");

print_architecture_details(&ia32);
print_architecture_details(&x86_64);
print_architecture_details(&armv8_4k);
print_architecture_details(&armv8_16k);
print_architecture_details(&armv8_64k);

// Summary comparison table
printf("\n");
printf("    SUMMARY COMPARISON\n");
printf(".\n\n");

printf("%-30s %-15s %-15s %-15s\n", "Architecture", "Address Bits",
"Page Size", "Levels");
printf("-----\n");

printf("%-30s %-15d %-15s %-15d\n", ia32.name, ia32.address_bits, "4
KB", ia32.levels);
printf("%-30s %-15d %-15s %-15d\n", x86_64.name, x86_64.address_bits,
"4 KB", x86_64.levels);
printf("%-30s %-15d %-15s %-15d\n", armv8_4k.name,
armv8_4k.address_bits, "4 KB", armv8_4k.levels);
printf("%-30s %-15d %-15s %-15d\n", armv8_16k.name,
armv8_16k.address_bits, "16 KB", armv8_16k.levels);
printf("%-30s %-15d %-15s %-15d\n", armv8_64k.name,
armv8_64k.address_bits, "64 KB", armv8_64k.levels);
printf("\n");
}

int main() {
    compare_architectures();
    return 0;
}

```

Tugas Praktikum Main Memory

Petunjuk Umum

1. Kerjakan semua tugas secara individu
2. Setiap tugas harus disertai dengan:
 - Source code yang berfungsi dengan baik
 - Screenshot hasil eksekusi
 - Analisis dan penjelasan
3. Kumpulkan dalam format:

```
NIM_Nama_TugasPraktikumMemory.zip
├── laporan.pdf
├── src/
│   ├── tugas1_address.c
│   ├── tugas2_allocator.c
│   ├── tugas3_paging.c
│   ├── tugas4_multilevel.c
│   └── tugas5_swapping.c
└── screenshots/
    ├── tugas1_output.png
    ├── tugas2_output.png
    └── dst...
```

Tugas 1: Analaisis Memory Layout

Deskripsi

Modifikasi program `address_demo.c` untuk menganalisis memory layout dengan lebih detail.

Yang Harus Dikerjakan

A. Modifikasi Program

Tambahkan fitur-fitur berikut pada program:

1. **Array Allocation:** Buat array statis dan dinamis, tampilkan alamatnya
2. **Multiple Functions:** Buat 3 fungsi berbeda, tampilkan alamat setiap fungsi
3. **Recursive Function:** Buat fungsi rekursif dan tampilkan alamat stack di setiap level
4. **String Literals:** Tampilkan alamat string literals

Contoh Output yang Diharapkan:

```
=== ENHANCED MEMORY LAYOUT ===
```

1. Text Segment:
main() address: 0x...

```
func1() address: 0x...  
func2() address: 0x...
```

2. Data Segment:

```
static_array[0] address: 0x...  
string_literal address: 0x...
```

3. Heap:

```
dynamic_array[0] address: 0x...  
malloc block 1: 0x...  
malloc block 2: 0x...
```

4. Stack (Recursive Calls):

```
Level 0 local_var: 0x...  
Level 1 local_var: 0x...  
Level 2 local_var: 0x...
```

B. Analisis

Jawab pertanyaan berikut dalam laporan:

1. Berapa jarak (dalam bytes) antara:

- Stack dan Heap?
- Text segment dan Data segment?
- Alamat fungsi satu dengan lainnya?

2. Jalankan program 5 kali, catat alamat-alamat yang muncul:

- Alamat mana yang selalu sama?
- Alamat mana yang berubah?
- Jelaskan mengapa terjadi perbedaan tersebut (hint: ASLR - Address Space Layout Randomization)

3. Bandingkan dengan memory map dari `/proc/[pid]/maps`:

- Apakah alamat yang Anda dapatkan sesuai dengan range di memory map?
- Sebutkan permission (rwx) dari setiap segment

Tugas 2: Implementasi Memory Allocator

Deskripsi

Kembangkan program memory allocator dengan fitur tambahan dan lakukan analisis performa.

Yang Harus Dikerjakan

A. Implementasi Next-Fit Algorithm

Tambahkan algoritma **Next-Fit** pada program `memory_allocator.c`:

- Next-Fit: Seperti First-Fit, tapi mulai pencarian dari posisi alokasi terakhir

```
int next_fit(int process_id, int size) {
    // TODO: Implementasi Next-Fit
    // Gunakan variabel global untuk menyimpan posisi terakhir
}
```

B. Fitur Fragmentasi

Tambahkan fungsi untuk menghitung fragmentasi:

```
void calculate_fragmentation() {
    // Hitung:
    // 1. Total free space
    // 2. Largest free block
    // 3. Number of free holes
    // 4. External fragmentation ratio
}
```

C. Benchmark Testing

Buat test case untuk membandingkan 4 algoritma:

```
void benchmark_test() {
    // Test case:
    // 1. Allocate: P1(100), P2(200), P3(300), P4(250), P5(150)
    // 2. Deallocate: P2, P4
    // 3. Allocate: P6(180), P7(220), P8(100)
    // 4. Hitung waktu eksekusi dan fragmentasi untuk setiap algoritma
}
```

D. Analisis (Dalam Laporan)

1. Buat tabel perbandingan:

Algoritma	Waktu Eksekusi (µs)	Fragmentasi (%)	Failed Allocations
First-Fit			
Best-Fit			
Worst-Fit			
Next-Fit			

2. Analisis:

- Algoritma mana yang paling cepat? Mengapa?

- Algoritma mana yang paling efisien dalam mengurangi fragmentasi?
- Dalam situasi apa setiap algoritma lebih unggul?

3. Buat grafik visualisasi memory allocation untuk setiap algoritma

Tugas 3: Advanced Paging System

Deskripsi

Kembangkan sistem paging dengan page replacement algorithm dan statistik.

Yang Harus Dikerjakan

A. Implementasi Page Replacement - FIFO

Modifikasi `paging_simulator.c` untuk menangani situasi ketika semua frame penuh:

```
int fifo_page_replacement(int new_page) {  
    // TODO: Implementasi FIFO  
    // 1. Cari page yang paling lama di memory  
    // 2. Evict page tersebut  
    // 3. Load page baru  
    // 4. Update page table  
}
```

B. Implementasi Page Replacement - LRU

Tambahkan algoritma LRU (Least Recently Used):

```
int lru_page_replacement(int new_page) {  
    // TODO: Implementasi LRU  
    // 1. Track waktu akses setiap page  
    // 2. Cari page yang paling lama tidak diakses  
    // 3. Evict page tersebut  
}
```

C. Statistik dan Analisis

Tambahkan fungsi untuk menghitung:

```
typedef struct {  
    int total_accesses;  
    int page_faults;  
    int page_hits;  
    float hit_ratio;  
    int disk_writes; // untuk dirty pages  
} PageStatistics;
```

```
void display_statistics(PageStatistics *stats) {
    // Tampilkan statistik
}
```

D. Test Scenario

Jalankan reference string berikut dengan 4 frames:

```
Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0,
1
```

Bandingkan hasil FIFO vs LRU:

- Jumlah page fault
- Hit ratio
- Disk writes (untuk dirty pages)

E. Analisis (Dalam Laporan)

1. Buat tabel trace untuk setiap algoritma:

Step	Reference	Frame 0	Frame 1	Frame 2	Frame 3	Fault?
1	7					
2	0					
...	...					

2. Hitung Belady's Anomaly:

- Jalankan dengan 3, 4, dan 5 frames
- Apakah page fault berkurang seiring bertambahnya frame?

3. Analisis performa:

- Algoritma mana yang lebih baik?
- Kapan FIFO lebih baik dari LRU?

Tugas 4: Multi-Level Page Table

Deskripsi

Implementasikan dan bandingkan berbagai struktur page table.

Yang Harus Dikerjakan

A. Three-Level Page Table

Kembangkan three-level page table system:

```
// Configuration: 16 + 8 + 8 + 12 = 44 bits (untuk demo)
#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET_BITS 12

typedef struct {
    int frame_number;
    int valid;
    int read;
    int write;
    int execute;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

// Implementasi:
int translate_3level(unsigned long virtual_addr);
void allocate_3level(unsigned long virtual_addr, int frame);
void calculate_memory_overhead();
```

B. Inverted Page Table (8 poin)

Implementasikan inverted page table:

```
typedef struct {
    int process_id;
    int page_number;
    int valid;
} InvertedPageEntry;

InvertedPageEntry inverted_table[NUM_FRAMES];

int search_inverted_table(int pid, int page_num);
void insert_inverted_table(int pid, int page_num, int frame);
```

C. Analisis Perbandingan (Dalam Laporan)

1. Memory Overhead Comparison:

Structure	Entries	Size per Entry	Total Memory	Space Efficiency
Single-Level				
Two-Level				
Three-Level				
Inverted				

2. Translation Speed:

- Hitung jumlah memory access untuk setiap struktur
- Berapa pengaruh TLB (Translation Lookaside Buffer)?

3. Kasus Penggunaan:

- Struktur mana yang cocok untuk sistem 32-bit vs 64-bit?
- Kapan inverted page table lebih efisien?

Tugas 5: Swapping Mechanism

Deskripsi

Simulasikan mekanisme swapping dengan berbagai strategi.

Yang Harus Dikerjakan

A. Basic Swapping Implementation

Buat program untuk mensimulasikan swap in/out:

```
typedef struct {
    int pid;
    int size;
    int priority;
    int in_memory;
    int swap_count;
    time_t last_access;
} Process;

void swap_out(Process *p);
void swap_in(Process *p);
void select_victim(); // Pilih proses untuk di-swap out
```

B. Swap Strategy Implementation

Implementasikan 3 strategi victim selection:

1. **FIFO**: Swap out proses yang paling lama di memory
2. **LRU**: Swap out proses yang paling lama tidak diakses

3. **Priority-based**: Swap out proses dengan prioritas rendah

C. Simulation Scenario

Test dengan skenario:

- Memory fisik: 1024 MB
- 10 proses dengan ukuran berbeda (50-200 MB)
- Total kebutuhan memory: 1500 MB
- Simulasi akses random selama 100 time units

D. Analisis (Dalam Laporan)

1. Metrik Performa:

Strategy	Swap Out Count	Swap In Count	Total Swap Time	Avg Response Time
FIFO				
LRU				
Priority				

2. Analisis:

- Strategi mana yang meminimalkan thrashing?
- Bagaimana ukuran memory mempengaruhi performa?
- Kapan swapping lebih baik daripada killing process?

Tugas 6: Architecture Analysis

Deskripsi

Analisis arsitektur memori pada sistem nyata.

Yang Harus Dikerjakan

A. Intel Architecture Analysis

Jalankan command berikut pada sistem Ubuntu:

```
# 1. Cek informasi CPU
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"

# 2. Cek page size
getconf PAGE_SIZE

# 3. Cek TLB info
cpuid | grep -i tlb

# 4. Cek cache hierarchy
lscpu | grep -i cache
```

Dokumentasikan dan analisis:

- Arsitektur prosesor (32-bit/64-bit)
- Page size yang digunakan
- Ukuran TLB
- Hierarchy cache (L1, L2, L3)

B. Memory Information

```
# 1. Total memory
free -h

# 2. Virtual memory info
vmstat 1 5

# 3. Memory map dari proses
cat /proc/self/maps

# 4. Swap usage
swapon --show
```

C. Analisis (Dalam Laporan)

1. Bandingkan spesifikasi sistem Anda dengan:

- Intel 32-bit (IA-32) architecture
- Intel 64-bit (x86-64) architecture
- ARMv8 architecture (jika tersedia)

2. Jelaskan:

- Berapa bit address space yang digunakan?
- Bagaimana implementasi multi-level paging?
- Berapa ukuran maksimum virtual memory?

Format Laporan

Struktur Laporan (Format PDF)

```
HALAMAN JUDUL
- Judul Praktikum
- Nama & NIM
- Kelas
- Tanggal
```

```
DAFTAR ISI
```

```
BAB I: PENDAHULUAN
```

- 1.1 Latar Belakang
- 1.2 Tujuan Praktikum
- 1.3 Spesifikasi Sistem

BAB II: DASAR TEORI

- 2.1 Memory Management
- 2.2 Contiguous Allocation
- 2.3 Paging
- 2.4 Page Table Structures
- 2.5 Swapping
- ... dst ...

BAB III: JAWABAN TUGAS PRAKTIKUM

Untuk setiap tugas:

- 3.X.1 Deskripsi Tugas
- 3.X.2 Source Code
- 3.X.3 Screenshot Output
- 3.X.4 Penjelasan Implementasi

BAB IV: ANALISIS DAN PEMBAHASAN

- 4.1 Analisis Tugas 1
- 4.2 Analisis Tugas 2
- ... dst ...

BAB V: KESIMPULAN DAN SARAN

- 5.1 Kesimpulan
- 5.2 Saran
- 5.3 Kendala yang Dihadapi

DAFTAR PUSTAKA

LAMPIRAN

- Source code lengkap
- Output tambahan

REFERENSI

1. Abraham Silberschatz, et al. "Operating System Concepts", 10th Edition
2. Andrew S. Tanenbaum. "Modern Operating Systems", 4th Edition